Axona Programmer Guide

A practical guide to building a decentralized pub/sub application on the Axona protocol. After reading this you should be able to ship a working chat, feed, inbox, or notification system without reading any other Axona code.

The guide assumes you already know JavaScript + Node + a browser. It does not assume any DHT / WebRTC / cryptography background – concepts are introduced where they are needed.

- **Protocol kernel**: @axona/protocol (v4.11.2)
- **Wire version**: 4.0 (`WIRE_VERSION`); kernel version 4.11.2 (`KERNEL_VERSION`)
- **Live network**: the 4.x line runs on **testnet** — `wss://testnet.axona.net`. The **production** bridges (`wss://bridge.axona.net` east, `wss://bridge-west.axona.net` west — a federated pair) are still on the 3.x line and do **not** interoperate with 4.x (the wire major partitions them). Build against testnet to use this surface.
- **Companion docs**:
  - Quick Start – five minutes to a working roundtrip; send a newcomer there first.
  - API Reference – every public symbol and its exact signature.

  What changed from the v2 line. v3.0.0 rebuilt identity, authorship, and addressing as three separate concerns (a breaking flag-day). If you learned Axona on the v2 guide, the things you must unlearn are: the single `deriveIdentity`, the `publisher`/`publishId` arguments, `sign: false`, bare string topics, and the "region-keyed / publisher-keyed / public" topic *modes*. All gone. The replacements are two identity factories (`createNodeIdentity`, `createAuthorIdentity`), descriptor topics (`{ region?, owner?, name, write? }`), and per-publish `signWith`.

  What changed in the 4.x line. The application API above is **unchanged** — the 4.x work is a routing/reliability rewrite underneath it: routing-only axonic-tree pub/sub, K-closest **cohort** replication (a killed message can't resurface and a publish survives the root churning out), a **cold-publish burst** so a freshly-joined node's first publish isn't lost, **nearest-replica reads** (`pull` answers from the closest replica holding the message), and a rebuilt **metrics** path. All transparent to your code; the reliability notes appear in §7 where they matter.

Table of contents

---

1. What is Axona?

Axona is a peer-to-peer mesh protocol where every participant is an equal node – there is no central server. Three things make it useful:

1. **Geographic routing.** Every node's ID and every topic's ID begins with a one-byte region prefix (a Google S2 cell). Lookups and message placement are XOR-routed in a way that strongly favors same-region neighbors, so a topic pinned to a region naturally clusters on nodes in that region.
2. **Signed authorship.** Every published message is signed by an *author key* (Ed25519). A reader can prove who authored a message, that it was not tampered with, and – if the author chose to be durable – recognize the same author later.
3. **Replicated cached delivery.** Each topic has a set of *axons* – the K closest peers to the topic ID – that hold a bounded ring of recent messages. Late subscribers can replay history; live subscribers tail in real time.

The protocol kernel (`@axona/protocol`) is environment-neutral pure JS. It runs in browsers, Node, and in-process simulation. Deployed components ride on top of it:

Component	What it is	Where it runs
`axona-peer`	Browser SDK + reference client	Static HTML/JS, any web host
`axona-bridge`	WebRTC signaling broker + universal-hub peer	A Docker stack (bridge + Caddy + coturn) behind auto-TLS
`axona-relay`	Headless hosting node (stores + serves topics)	Node
`dht-sim`	Visualization + analysis simulator	Node, locally

The bridge is a signaling broker for WebRTC and a universal-fallback peer in the mesh – it is **not** a centralized message queue. Most messages flow directly between browsers over WebRTC data channels; the bridge is a WebSocket fallback when NAT prevents a direct connection, and a hub that helps mesh discovery on cold start. The protocol does not depend on the bridge for correctness.

The three separated concerns (read this once and it all fits)

The single most important idea in Axona v3 is that **connection, authorship, and addressing are three different things, each with its own primitive, and no one of them does another's job:**

Concern	The question it answers	Primitive	Lifetime
Connection	"Which node is this, and is it really it on the wire?"	**Node Identity** -> **Node ID**	ephemeral (fresh each run)
Authorship	"Who is accountable for this message, provably?"	**Author Identity** -> **Author ID**	your choice: per-run or durable
Addressing	"Where does this topic live? Is this a duplicate?"	**Topic ID** / **Message ID**	deterministic, identity-independent

Two consequences are the heart of the model:

- **Connection is not authorship.** The connection key is thrown away and re-minted each run (so being online is not trackable). The author key is the thing you keep (so your authorship is recognizable). Either can change without disturbing the other.
- **Addressing is not identity.** Where a message lives and whether it is a duplicate come from the topic and the content, not from who you are. Routing and dedup work for anonymous senders too.

Sections 4, 5, and 6 are one per concern. Internalize the table above and the rest is detail.

Address space

Every Node ID and every Topic ID is **264 bits = 66 lowercase hex characters**. The layout is the same for both: a one-byte region prefix followed by a 256-bit hash.

```
+--------------------------+----------------------------------------+
|  region byte             |  256-bit hash                          |
|  8 bits (top 2 hex)      |  64 hex chars                          |
+--------------------------+----------------------------------------+
```

- **Node ID** = `regionByte || SHA-256(pubkey)` – the region is the S2 cell of the node's location; the hash is of its connection public key.
- **Topic ID** = `regionByte || SHA-256(canonical({ owner, name, write }))` – see section 6.

Because node IDs and topic IDs share the same space, the XOR distance between a node and a topic is meaningful, and the router uses it to keep locality.

---

2. Setup

2.1 Install the kernel

```
mkdir my-axona-app && cd my-axona-app
npm init -y
npm install @axona/protocol@github:axona-net/axona-protocol#v4.11.2
```

You now have `node_modules/@axona/protocol/src/` with the full kernel.

2.2 The shape of a peer

A running peer is four objects:

```
import {
  AxonaPeer, AxonaDomain, NeuronNode,
  createNodeIdentity, createAuthorIdentity,
  KERNEL_VERSION,
} from '@axona/protocol';
import { webTransport } from '@axona/protocol/transport/web/index.js';

// 1. The CONNECTION identity (needs a location -> yields a Node ID).
const nodeIdentity = await createNodeIdentity({ lat: 38.0, lng: -77.0 });

// 2. A transport. In the browser, webTransport speaks WebRTC + a WS bridge.
const transport = webTransport({
  bridgeUrl: 'wss://testnet.axona.net',  // the 4.x line runs on testnet (prod is 3.x)
  identity:  nodeIdentity,               // the factory option is named `identity:`
});

// 3. The local routing state.
const node = new NeuronNode({
  id:  BigInt('0x' + nodeIdentity.id),
  lat: 38.0, lng: -77.0,
});
node.transport = transport;
```

```
// 4. The peer ties them together. Note: NO author key here -- authorship
//    is per-publish, never a constructor argument (see section 5).
const peer = new AxonaPeer({
  domain:       new AxonaDomain({ k: 20 }),
  node,
  nodeIdentity,
  transport,
});

await transport.start(nodeIdentity.id);
await peer.start();
console.log('connected; kernel', KERNEL_VERSION);
```

In Node you would use a node transport instead of `webTransport`, but the peer-construction shape is identical. The fastest way to see all of this working end to end is to read `apps/axona-minimal/` in the kernel repo – a complete ~70-line app that section 12 is based on.

2.3 Or just open the deployed peer

To watch Axona work without writing code, open https://axona.net in two browser tabs. Each tab is an independent peer. Type the same topic in both, publish from one, and the message arrives in the other.

---

3. Mental model

3.1 Peers and the mesh

Every participant is a **peer** – a long-lived process with:

- **A node identity**: a 264-bit Node ID and an Ed25519 connection keypair.
- **A synaptome**: an adaptive routing table of other peers, weighted by recency, latency, and usefulness.
- **A transport**: a way to talk to other peers. In the browser this is a composite of a WebRTC mesh (direct data channels) and a WebSocket fallback through the bridge.

Peers form a **mesh** – a graph of bidirectional channels. The mesh is *not* a global broadcast topology. Each peer directly knows only a handful of others; everything else is reached by routed forwarding.

The mesh is **self-healing**, and you don't manage it. A peer continuously keeps its routing table *complete* in two ways: it holds its few XOR-nearest "successor" peers (so routed delivery's last hop always lands on the right node) and a spread of longer-range links (so any target is reachable). When a neighbour drops, the replacement is almost always a peer it already knows of – so repair is a cheap local step, not a global search. As an app author there is nothing to configure: subscribe, publish, and let the mesh maintain itself underneath you.

3.2 Topics, axons, and replication

A **topic** is a small descriptor that hashes to a 264-bit Topic ID (section 6). When you `peer.pub(...)`, the protocol:

1. Resolves the descriptor to a Topic ID.
2. Finds the **K closest peers** to that Topic ID in XOR space (K defaults to 5).
3. Sends a publish frame to each. Each appends the message to its local replay cache for the topic and forwards it to any subscribers it tracks as children.

Those K peers are the topic's **axons**. They are not a privileged server set – any peer that happens to land in the K-closest set for a Topic ID acts as an axon for it, and the set rotates as nodes join and leave.

A `peer.sub(...)` registers the caller as a child of the topic's axons; from then on, publishes fan out to it. If it asks for `{ since: 'all' }`, the axon also replays its cache.

3.3 Geographic clustering

The region byte at the top of every ID means a topic pinned to `useast` gets a Topic ID beginning `0x89`, and `useast` nodes – XOR-close to that prefix – are naturally selected as its axons, where its subscribers are also likely to live. That minimizes cross-region hops. You pin a topic to a region by naming `region` in the descriptor (section 6).

3.4 The signed envelope

Every publish is wrapped in an envelope and delivered to subscribers as:

```
{
  msgId:        '<64-char hex>',   // SHA-256(authorId + message); dedup key
  ts:           1779306455550,     // ms epoch
  topic:        { region, owner, name, write },  // the SIGNED descriptor (object, not a string)
  message:      <any JSON value>,  // your opaque payload
  signerPubkey: '<64-char hex>',   // the Author ID; absent for anonymous
  signature:    'ed25519:<...>',   // absent for anonymous
}
```

`message` is opaque bytes to the protocol – string, number, object, array, anything JSON-serializable. The kernel JSON-encodes the whole envelope at the wire layer.

3.5 What the protocol secures (and what it does not)

Axona is **self-authenticating**: every guarantee is enforced by cryptography the peers carry themselves – no certificate authority, no central trust server.

- **Identity is provable, not asserted.** A Node ID's bottom 256 bits are `SHA-256(pubkey)`, and every connection runs a handshake that proves the peer holds the key and binds the proof to *that live connection* so it cannot be replayed onto another link.
- **Authorship is provable and self-contained.** Every accountable message carries its Author ID plus a signature over the message; receivers verify it and recompute the Message ID. A forwarder can never make one author's content look like another's.
- **Write policy is enforced at the storing node.** An owned topic (section 6) accepts a publish only if the signer is the owner (`WRITE_POLICY_VIOLATION` otherwise) – checked at ingress, before fan-out.
- **You can only act for yourself.** A peer subscribes only *itself*; nodes host only topics they are routing-responsible for; a `kill` is accepted only if signed by the original author.
- **Location is never disclosed by the protocol.** No message ever carries a region, coordinate, or Node ID. The region byte lives only inside the ephemeral Node ID, and it is never derived from the author key. (If your app wants to show a sender's region it must put that in the payload itself – a deliberate app choice, not a protocol field. The worked example in section 12 does exactly this.)

What it does not do: encrypt your payload (`message` is opaque – encrypt inside it end-to-end if you need confidentiality), and it does not decide *whether you trust* a given author – that is your app's policy (allowlist, web of trust, trust-on-first-use, or "accept everyone"). See the security changelog for the versioned record.

---

4. Node identity (the connection)

4.1 Creating one

```
import { createNodeIdentity } from '@axona/protocol';
```

```
const node = await createNodeIdentity({ lat: 38.0, lng: -77.0 });
// node = {
//   id:          'df27c92b...' (66-char hex Node ID),
//   pubkey:      Uint8Array(32),
//   pubkeyHex:   '<64 hex>',
//   privateKey: <Web Crypto Ed25519 CryptoKey>,
//   region:      { lat: 38.0, lng: -77.0 },
//   createdAt:   1779306455550,
//   sign, verify,
// }
```

The top byte of `node.id` is the S2 cell for `(lat, lng)`; the bottom 256 bits are `SHA-256(pubkey)`. This key authenticates the connection handshake, drives DHT routing, and is what `subscribe` rides on. **It never signs a published message** (key separation – section 5).

`createNodeIdentity` mints a fresh keypair every call. By default the key is extractable so it can be persisted; pass `{ extractable: false }` for an ephemeral browser identity that XSS cannot exfiltrate.

4.2 Ephemeral by default – and that is the point

A node identity is meant to be **thrown away and re-minted each run.** A restarted peer rejoins as a new node with a new Node ID. This is a privacy property: nobody can build a durable record of "this node was online again."

You *may* persist it for a stable address (`dumpIdentity` / `loadIdentity`, section 10), but you rarely want to – a stable Node ID means linkable connections. The thing you keep across sessions is your **author** key (section 5), not your node key.

4.3 Choosing a location

The location determines your region byte, which is a routing neighborhood. Three strategies:

Strategy	When
Hardcode `{ lat, lng }`	A server-side peer (relay, daemon, scraper) whose region is known.
Geolocate the user	`navigator.geolocation.getCurrentPosition`, falling back to a default region if denied.
Let the user pick	A region dropdown; use `regionCenter(name)` from the region module to turn a choice into `{ lat, lng }`.

The region module (`@axona/protocol`) ships the canonical region table and converters:

```
import { regionName, regionCode, resolveRegion, regionCenter } from '@axona/protocol';

regionName(0x89);        // -> 'useast'
regionCode('useast');    // -> 0x89 (137)
resolveRegion('0x89');   // -> 137   (accepts a name, '0x89', '137', or 137)
regionCenter('useast');  // -> { lat, lng } -- feed straight into createNodeIdentity
```

Your node region is independent of which regions' topics you address – you can publish to a topic pinned to any region regardless of where your node lives (section 6).

5. Author identity and signing (the authorship)

5.1 The author key has no location and no Node ID

```
import { createAuthorIdentity } from '@axona/protocol';

const me = await createAuthorIdentity();          // ephemeral author
// me = { kind: 'author', authorId: '<64 hex>', pubkey, pubkeyHex, privateKey, sign, verify }
```

`me.authorId` is your public **Author ID** – the value that appears on every message you sign (it is the `signerPubkey` field on the wire). An author identity is a keypair and nothing else: **no location, no region, no Node ID.** Authorship is not a place.

5.2 Durability is the author's choice – the only persistence decision most apps make

```
await createAuthorIdentity();                              // ephemeral: one-shot, unlinkable
await createAuthorIdentity({ persistAs: 'me' });           // durable: load-or-create + save to localStorag
await createAuthorIdentity({ persistAs: 'me', store }); // durable: custom { get, set } store
```

- **Ephemeral** – minted fresh, unlinkable, gone when the tab closes. Good for genuinely throwaway posting.
- **Durable** (`persistAs`) – on first call it mints and saves; on later calls it loads the saved key. The same Author ID survives reloads, so readers recognize you across sessions and you can `kill` your own content later. `persistAs` is a *local storage label*, not a network name.

In the browser `persistAs` uses `localStorage` by default; pass `store` (any `{ get, set }`) to control where it lands. A persisted key is forced extractable so it can be serialized.

5.3 An app may hold several authors – or none

A peer holds **no default author.** Authorship is supplied at the one moment it is used – signing a publish – via `signWith`. This is deliberate:

- A subscribe-only peer needs no author at all.
- A peer can run several authors (personas) at once, choosing which signs each message.

```
const alice = await createAuthorIdentity({ persistAs: 'alice' });
const bot   = await createAuthorIdentity({ persistAs: 'bot' });

await peer.pub({ name: 'lobby' }, 'hi from alice', { signWith: alice });
await peer.pub({ name: 'lobby' }, 'beep boop',     { signWith: bot });
```

5.4 Every publish names its signer – there is no default

This is a hard rule. Each `peer.pub` (and `kill`) takes a `signWith`:

```
import { ANONYMOUS } from '@axona/protocol';

await peer.pub({ name: 'lobby' }, msg, { signWith: me });         // signed by `me`
await peer.pub({ name: 'lobby' }, msg, { signWith: ANONYMOUS });  // explicitly anonymous (unsigned)
await peer.pub({ name: 'lobby' }, msg);                           // ERROR: PUBLISH_NO_PUBLISH_IDENTITY
```

Omitting a signer is an **error**, never silent anonymity. Anonymity must be asked for by name (`ANONYMOUS`). And the node key never signs a publish – even if you wanted it to, you would have to pass it explicitly as the signer, which is discouraged.

An anonymous message has no provable author, so it carries no `signerPubkey`, its Message ID is `SHA-256(message)` alone, and it **cannot be killed** (there is no author to authorize the retraction).

5.5 Verifying what you receive

The network already drops spoofed-signature traffic at a topic's ingress, but verifying what you consume is good practice if your app gates on identity:

```
import { verifyEnvelope } from '@axona/protocol';

await peer.sub({ name: 'lobby' }, async (env) => {
  if (env.signature) {
    const res = await verifyEnvelope(env);   // returns { ok, reason, signed }
    if (!res.ok) { console.warn('rejected', env.msgId, res.reason); return; }
  }
  appendToFeed(env);
}, { since: 'all' });
```

> `verifyEnvelope` returns a **result object**, not a boolean – test `.ok`. `if (!(await verifyEnvelope(env)))` is always false (an object is truthy), so that guard would silently never reject anything.

`verifyEnvelope` proves the message came from `env.signerPubkey` and was not tampered with. It does **not** decide whether you *trust* that Author ID – that is your application's call.

---

6. Topics and addressing

This section is the home for everything about topic IDs.

6.1 A topic is a descriptor, not a string

A topic is a small **descriptor**, and the Topic ID is a hash of it with a one-byte region prefix:

```
topicId = regionByte || SHA-256(canonical({ owner, name, write }))   // 33 bytes = 66 hex chars
```

Field	Meaning
`region`	a real geographic cell; becomes the leading byte of the ID
`owner`	an **Author ID** (a publish key), or absent
`name`	the human label – `"lobby"`, `"profile"`
`write`	`"open"` or `"owner"` – **defaults by whether `owner` is set** (6.3)

Because the ID is a pure function of those fields, **anyone who knows the fields computes the identical ID** – no registry, no coordination.

6.2 The `region` field: name or code, or omit it

`region` accepts a region **name**, **hex string**, **decimal string**, or **number** – all normalize to the same region byte:

```
{ region: 'useast', name: 'lobby' }   // name
{ region: '0x89',   name: 'lobby' }   // hex string   --\  same Topic ID
{ region: 137,      name: 'lobby' }   // number       --/  (leading byte 0x89)
```

`'useast' == '0x89' == 137`. **Omit `region`** and it defaults to the *publisher's own node region* (the top byte of its Node ID) – two co-located peers converge on the same topic with no region named. To share a topic across regions, name the region explicitly. An unknown region throws.

Region is **never** derived from the author key (an Author ID has no location), and there is **no global region** – a topic the whole world reads picks one real region (a deliberate, app-visible placement) or is sharded across regions at the app layer.

6.3 `write` defaults by whether there is an `owner`

Descriptor	Resolved `write`	Meaning
`{ region, name }` (no owner)	`open`	open lobby – anyone publishes
`{ region, owner, name }` (no write)	`owner`	owned – only the owner publishes
`{ region, owner, name, write: 'owner' }`	`owner`	same ID as the row above
`{ region, owner, name, write: 'open' }`	`open`	owner-namespaced inbox – anyone publishes
`{ region, name, write: 'owner' }` (no owner)	`open`	no owner -> `write` ignored

Two rules to remember:

- **No `owner` -> the topic is open; `write` is ignored.** You cannot have an owner-only topic with no owner.
- **An `owner` -> `write` defaults to `'owner'`** (the safe default). So `{owner, name}` and `{owner, name, write:'owner'}` are the *same topic*, and forgetting `write` can never silently make an owned feed world-writable. Pass `write:'open'` explicitly only when you want an owner-namespaced topic anyone may post *to* (an inbox).

6.4 The three shapes

```
import { createAuthorIdentity } from '@axona/protocol';
const me = await createAuthorIdentity({ persistAs: 'me' });
```

Open lobby – `{ region, name }`. Anyone publishes (each message self-signed or anonymous), anyone subscribes.

```
await peer.pub({ region: 'useast', name: 'lobby' }, 'hello', { signWith: me });
await peer.sub({ region: 'useast', name: 'lobby' }, (env) => {
  console.log(env.signerPubkey, env.message);
}, { since: 'all' });
```

Author feed (profile / broadcast) – `{ region, owner, name }`, write defaults to `owner`. Only messages signed by the owner key are accepted; the storing node recomputes the ID from the signed descriptor and rejects anything where `signer !== owner`. A profile cannot be spoofed even by talking directly to a storage node.

```
const feed = { region: 'useast', owner: me.authorId, name: 'profile' };
await peer.pub(feed, { status: 'online' }, { signWith: me });   // only `me` may write
```

Author inbox (post *to* someone) – `{ region, owner, name, write: 'open' }`. The owner namespaces it, but anyone may post (signed as themselves); the owner reads. "Reach author X" = put X's Author ID as `owner`, sign with your own key.

```
await peer.pub(
  { region: 'useast', owner: aliceAuthorId, name: 'inbox', write: 'open' },
  { from: me.authorId, body: 'hi alice' },
  { signWith: me });
```

Reading is always open. But the Topic ID is *deterministic, not secret*: a stranger cannot compute an owned feed's ID without the owner's Author ID, so the owner shares the ID (or the descriptor) with readers.

6.5 Generating a Topic ID to share

`deriveTopicId(descriptor)` returns the 66-hex Topic ID – the same function the peer uses internally. It works for every topic kind. This is the **read / subscribe handle** you hand to someone:

```
import { deriveTopicId } from '@axona/protocol';

const lobbyId = await deriveTopicId({ region: 'useast', name: 'lobby' });
const feedId  = await deriveTopicId({ region: 'useast', owner: me.authorId, name: 'profile' });
// -> e.g. "89a1b2c3..."  (66 hex; leading "89" is the region byte)
// share lobbyId / feedId out-of-band (URL, QR, message)
```

`sub`, `pull`, and `metrics` accept **either** a descriptor object **or** a bare 66-hex Topic ID:

```
await peer.sub(lobbyId, onMsg, { since: 'all' });   // subscribe by ID
await peer.pull(someMsgId, { topic: feedId });      // pull by ID
await peer.metrics(feedId);                          // metrics by ID
```

6.6 Publishing needs the descriptor, not the ID

`pub` (and the owner op `kill`) requires the **descriptor** – passing a bare ID throws `PUBLISH_INVALID_TOPIC`. This is by design:

- The storing node must recompute the ID from `{ region, owner, name, write }` to enforce the write policy (`signer === owner`). A Topic ID is a hash; it cannot reveal its `owner`, so the ID alone cannot prove you are authorized to write. (If it could, anyone who learned an owned feed's ID could post to it.)
- For an open topic the publish descriptor is just `{ region, name }` – tiny and human-meaningful, so "share the topic so someone can post" means sharing those two fields. The owner of an owned topic already holds the full descriptor.

In short: share the ID for reading; share the descriptor for writing.

6.7 Multiple publishers on one owned topic – proposed, not built

Authorizing additional publishers on an owned topic is **not implemented**: `write: 'owner'` means exactly the one `owner` key. The roadmap design is an owner-signed writer roster (the topic still commits to one `owner` in its ID; the owner publishes a signed `{ topicId, writers, seq }` record; storing nodes accept a publish if the signer is the owner or is in the latest roster, and the Topic ID never changes). Do not build against it yet.

6.8 Quick reference

	Open topic	Owned topic
Descriptor	`{ region, name }`	`{ region, owner, name }` (write -> `owner`)
Who may publish	anyone (self-signed / anonymous)	only the `owner` key (node-enforced)
Who may subscribe	anyone	anyone given the ID / descriptor
`region` forms	name, `'0x89'`, `137`, or omit -> node region	same
Generate ID to share	`deriveTopicId({ region, name })`	`deriveTopicId({ region, owner, name })`

	Open topic	Owned topic
Read by ID	`sub/pull/metrics` accept the 66-hex ID	same
Publish by ID	no – needs `{ region, name }`	no – needs the descriptor (write policy)
Multiple writers	n/a	proposed (owner-signed roster), not built

---

7. Publish and subscribe

7.1 Publish

```
const msgId = await peer.pub(topicDescriptor, message, { signWith });
// returns the envelope's msgId (64-char hex content hash)
```

- `topicDescriptor` is `{ region?, owner?, name, write? }` – a bare ID is rejected (6.6).
- `message` is any JSON-serializable value.
- `signWith` is an author identity, or the `ANONYMOUS` sentinel. **Required** (5.4).

A publish above the WebRTC-interoperable floor (**16 KiB** on the serialized envelope) throws `PUBLISH_PAYLOAD_TOO_LARGE` immediately rather than being silently dropped mid-mesh. Chunk large payloads with `@axona/protocol/std/chunk` (`publishChunkedBytes`), or – better for images and documents – publish a small content reference (hash + size + mime) and transfer the bytes out-of-band. A non-serializable message (circular ref, BigInt) throws `PUBLISH_INVALID_MESSAGE`.

> **`std/chunk` is reliable by default (kernel 3.6.0).** `publishChunkedBytes` paces the chunks and then **verifies what the mesh cached and re-publishes any gaps** – `peer.pub` is fire-and-forget into the transport buffer, so a fast burst would otherwise drop chunks before they durably cache and a *reload* subscriber would never reassemble. You no longer pick a throttle value. The receiver (`receiveChunkedBytes`) reassembles, or rejects with the missing indices on timeout – it never hangs.
>
> **Delivery, and why you don't ack (4.x).** A publish routes to the topic's K-closest **cohort** and is replicated across it, so it survives the closest node churning out. There is deliberately **no delivery ack to the publisher** — the transport identity is unlinked from the author identity, so the network can't (and won't) tell you "who received it where." Two automatic mechanisms cover the gaps so you don't hand-roll retries: a freshly-joined ("cold") node re-sends its first publishes a few times over ~1 s (the **cold-publish burst**, v4.11.0) so a not-yet-warm routing table doesn't strand them, and a background retry re-sends a recent publish until the publisher sees its own `msgId` land as a root. This is why §13.3's "publish before the mesh forms" pitfall is now a soft edge, not a cliff.

7.2 Subscribe

```
const sub = await peer.sub(topic, (envelope) => {
  if (envelope.deleted) { ui.remove(envelope.msgId); return; }  // a kill arrived
  ui.render(envelope.message);
}, { since: 'all' });

// later
await sub.stop();
```

- `topic` is a descriptor **or** a bare Topic ID (6.5).
- The handler receives the full envelope (3.4), or a delete marker `{ msgId, topic, deleted: true }` when a message is retracted – branch on `envelope.deleted`.

- `peer.sub` returns a `Subscription`; call `.stop()` to leave (or `peer.unsub(topic)` – 7.6).

7.3 Replay vs live tail (`since`)

`since`	Behavior
omitted	Live tail only – no cached messages.
`'all'`	Replay everything in the axons' caches, then live tail.
`'latest'`	Replay the single newest retained message (the current value, regardless of age), then live tail.
`<number>`	Replay messages newer than this ms-epoch timestamp, then live tail.

For chat / feed UX you almost always want `since: 'all'` – new subscribers expect history.

7.4 Queue size and hold time

Each topic's replay queue is bounded:

- **Max messages** – default **1024**, with a **16 MB** per-topic byte cap; eviction honors whichever binds. Order comes from the signed `seq`, so every replica evicts the same message and caches stay consistent.
- **Hold time** – a message past its hold expires and stops being delivered or pulled. A `pull` or `touch` slides the hold forward (bounded by an absolute ceiling), so reads keep a hot message alive but cannot pin it forever.
- **Per-publisher quota (open topics)** – on an anyone-may-publish topic one publisher can hold at most a bounded fraction of the queue, so a single flooder cannot evict everyone else. Owner-only topics are not quota'd.

The cache is in-memory and vanishes when an axon disconnects. A topic with no subscribers and no host for a short grace window is swept – so without a hosting node (7.9), replay history can be lost even before its hold elapses.

7.5 Subscriptions are soft-state leases

A subscription is *soft state*. You call `peer.sub()` **once**; the kernel re-announces it to the topic's current axons on a ~10s tick, and an axon forgets a subscriber it has not heard from in ~30s. The 3x headroom means a transient hiccup does not evict you, and a peer that goes silent and comes back re-subscribes automatically on the next refresh (with `since`-replay backfilling anything still in the hold window).

For your application: nothing to manage. You never renew subscriptions yourself.

7.6 Lifecycle: unsub, kill

Both are self-authenticating – authority is proven by the same author key, no gatekeeper. `kill` names its author via `signWith`.

```
// Leave a topic (stops all your local subs for it; sends the network unsub).
await peer.unsub({ region: 'useast', name: 'lobby' });

// Retract a message YOU authored ("unsend"). Accepted only if signed by the
// SAME author key that signed the original.
await peer.kill({ region: 'useast', name: 'lobby' }, msgId, { signWith: me });
```

> **Removed in v4.3.0: `touch` and `unpub`.** `kill` is now the single retraction primitive. `unpub` (clearing a whole owned-topic queue) is gone — a queue empties naturally as messages age out at the 48 h ceiling. `touch` (a hold-time keep-alive) is gone too — keep a still-relevant message current

by **re-publishing** it (an upsert that resets the hold *and* the ceiling; see §7.7). `peer.touch()` remains callable as a no-op for source compatibility.

Honesty note: `kill` is **best-effort redaction, not a cryptographic un-send** – a subscriber that already has the plaintext can keep it, and an offline one may never see the purge. An anonymous message cannot be killed.

7.7 Message hold time: re-publish refresh, touch, and the ceiling

A message's hold (expiry) is set off its signed `ts` when a root stores it, clamped to receive-time and bounded by a 48h ceiling. Three things to know:

Re-publishing the same message refreshes it (an upsert). The msgId is `SHA-256(author + message)` (no `ts`/`seq`/nonce – see 3.4), so re-publishing the identical payload yields the **same msgId**. As of kernel v3.3.0 the storing node **upserts**: it replaces the older copy with the new one, so there's always exactly one entry per msgId – the newest – and the new publish sets a **fresh hold and a fresh 48h ceiling**. So re-publishing *does* keep a message alive (indefinitely, if you keep doing it). It is **delivered once**: subscribers who already received that msgId are not re-notified, and late subscribers get it once via replay-on-subscribe. (Different authors of identical text have **different** msgIds and are not collapsed; add a nonce to `message` for an independently-addressable copy by the same author.)

`pull` also refreshes, but cannot reset the ceiling. A `pull` slides the hold forward as a side effect of the read (locally, on the serving replica), clamped to the entry's **current** `ceilingAt`. So a pulled message lives a little longer without re-notifying; re-publish when you want a fresh ceiling and to re-establish the content as the newest entry. (Pre-v4.3.0 `touch` did this deliberately; it's removed — re-publish instead.)

The ceiling. An entry can never outlive its `ceilingAt = ts + 48h` via `pull`. A re-publish sets a new `ts`, hence a new ceiling – which is why re-publishing can extend indefinitely while a pull-refresh cannot.

With `kill`: because there is one entry per msgId, a single `kill(topic, msgId, { signWith })` retracts it – no stale copy is left behind. The tombstone that blocks re-acceptance is finite-lived, and since the msgId is topic-independent, the same content published to two different topics must be killed in each.

7.8 Pull and metrics

```
// Fetch one envelope by its content hash (e.g. from a reshare link), or the
// latest with a null msgId. Returns null on a cache miss / expired message.
const env    = await peer.pull(msgId, { topic: feedId, timeoutMs: 1000 });
const latest = await peer.pull(null,  { topic: feedId });

// Coarse, best-effort topic state. Works for ANY topic - open or owned.
const m = await peer.metrics(feedId, { timeoutMs: 1500 });
// { current_count, seq, subscribers, bytes, publishes, ts, signer, cohortSize, stale }
```

`pull` is for "did I miss this one?", not durable storage. **Nearest-replica reads (v4.11.1–v4.11.2):** a pull is answered by the first replica it reaches — a cohort member, a child relay, or a `host()` node — not necessarily the root. `pull(msgId)` is *exact* (the content hash pins the message); **pull-latest** (`null` msgId) gives whatever newest that replica holds — *recent, eventually-consistent* — which keeps a heavily-polled "current value" read from hammering the root. Need the strict newest? Pull a specific `msgId`.

`metrics()` (v4.3.0) is a one-shot read of the latest **published** metric snapshot — a topic's roots publish a signed snapshot to `metricTopic(T)` every ~20 s, and `metrics()` subscribes to it briefly and aggregates across the cohort (`stale:true` if no publisher roots the topic yet). It works for **any** topic, open or owned — an owned topic's metrics are public too. `current_count` is the live retained count; `seq` is the dense message counter (total events ever, kills included); `subscribers` is the topic-wide total (summed across the cohort); `cohortSize` is how many roots reported. For UX, not billing.

For a live count, prefer `sub(metricTopic(T), …)` directly (next section): one standing subscription, every update pushed to you, plus a rolling history. `metrics()` is the convenience one-shot.

7.8b Watching metrics live (`metricTopic`)

For a continuously-displayed count, **subscribe to the topic's metrics.** A topic's primary root publishes a signed snapshot to a *derived, open* topic every ~20 s — for **both open and owned** data topics; you compute that topic with `metricTopic()` and `sub()` it like any other:

```
import { deriveTopicId, metricTopic } from '@axona/protocol';

const lobbyId = await deriveTopicId({ region: 'useast', name: 'lobby' });
await peer.sub(metricTopic(lobbyId), (env) => {
  const m = JSON.parse(env.message);   // { topic, ts, by, signer, current_count, seq, subscribers, byt
  showRoomCount(m.subscribers, m.current_count);
}, { since: 'all' });                  // latest snapshot + a rolling ~48 h trend
```

You pay one subscription instead of a poll, you get every update pushed to you, and because each snapshot is an ordinary retained message that ages out at the 48 h hold ceiling, `since:'all'` hands you a **rolling ~48 h history** — enough to plot a trend line, free.

Three things to keep in mind:

- **Advisory, not authoritative.** The metric topic is *open* (anyone can publish to it), so treat a snapshot as a hint. If you only trust your own relays, check `env.signerPubkey` against their keys.
- **Owned topics included (v4.3.0).** An owned (`write:'owner'`) topic's metrics are published to its (open) metric topic too — so anyone can subscribe to an owned topic's activity metrics without owning it. The *messages* stay write- gated; only the activity counts are public.
- **You don't publish these — relays do.** As an app you only ever *read* metric topics. The publish side runs on infrastructure (see §7.9 / the relay).

7.9 Hosting a topic without subscribing (`host` / `unhost`)

A relay or archive node wants to *store and serve* a topic without consuming it. That is `host`, decoupled from `sub` on purpose – hosting is "I will serve this for others," subscribing is "I want to receive this."

```
await peer.host();                                  // host my keyspace neighborhood
await peer.host({ region: 'useast', name: 'lobby' }); // host one specific topic
await peer.unhost({ region: 'useast', name: 'lobby' });
await peer.unhost();                                 // stop keyspace hosting
```

`host()` with no argument makes the node a willing root for whatever topics land near its Node ID – this is how `axona-relay` provides durable history. `host(topic)` pins one topic. Hosting registers no handler and delivers nothing to your app.

---

8. Direct messaging

Not all peer-to-peer communication is pub/sub. For 1:1 messages, `AxonaPeer` exposes a direct API. There is one handler per peer; dispatch on payload shape if you want multiple message kinds.

```
// Request/response -- Alice asks Bob, awaits his handler's return value.
const reply = await alice.send(bobNodeIdHex, { kind: 'ping', body: 'hello' });

// Fire-and-forget.
await alice.notify(bobNodeIdHex, { kind: 'typing', user: 'alice' });
```

```
// Bob's single handler -- invoked for every inbound direct message.
bob.onMessage(async (senderId, message) => {
  switch (message?.kind) {
    case 'ping':   return { reply: 'pong' };      // becomes Alice's send() resolution
    case 'typing': console.log(`${senderId} typing`); return;
    default:       return { error: 'unknown kind' };
  }
});
```

- The target is a **66-char hex Node ID** (the value `node.id` / `peer.getNodeId()` returns), and the peer must already have a channel to it. Routing to a peer you have no channel with is the job of higher layers.
- `senderId` is set by the transport at the receiving end; trust it for routing, but verify application-layer identity yourself if it matters.
- `onMessage` is **single-handler** – calling it again replaces the previous handler. Dispatch on `message.kind` for multiple kinds.
- `send`/`notify` do **not** sign the way `pub` does. For end-to-end authentic DMs, sign your payload yourself and verify on receipt.

---

9. Mesh introspection

9.1 Who's online

```
peer.peers();   // -> ['<66 hex>', ...] node IDs currently in the synaptome

const off1 = peer.onPeerJoin((peerId) => console.log('hi', peerId));
const off2 = peer.onPeerLeave((peerId) => console.log('bye', peerId));
off1(); off2();   // each returns an unsubscribe function
```

`peers()` is the synaptome – everyone in a small mesh, a constant-sized subset in a large one.

9.2 Lookup

```
const result = await peer.lookup(BigInt('0x' + targetHex));
// { found, hops, time, path }
```

An iterative XOR-routed walk; warms the local synaptome as a side effect.

9.3 Health

```
peer.health();
// {
//   nodeId, synaptomeSize, peers: [...], subscriptions,
//   axonRoles: [{ topic, isRoot, children, cacheSize }],
//   hosting, wireVersion, started,
//   transport: { boundCount, meshChannels, meshOpen, meshBound, bridgeState } | null,
//   meshDegraded,   // true => channels OPEN but NOT yet authenticated
// }
```

`meshDegraded` is the routing-truth signal: it distinguishes "channels are open" from "channels are open *and* carrying authenticated routing." A single true tick is a normal mid-handshake transient; a value that stays true across several polls means the mesh looks connected but is not actually routing. `boundCount` / `meshBound` are the honest usable-peer counts. Cheap; safe to poll on a UI tick.

9.4 Logs, errors, upgrades

```
const off = peer.onLog('warn', (msg, ctx) => { /* 'debug'|'info'|'warn'|'error' */ });
peer.onError((err) => { /* background AxonaError */ });
peer.onUpgradeRequired((err) => { /* show an upgrade banner */ });
```

`onLog` takes the **level first**, then the handler, and returns an unsubscribe. The kernel forwards its security drop-path logs (bad signature, stale, oversize, write-policy violation, unauthorized kill, …) to `onLog` so you can surface them.

---

10. Persistence

10.1 The author key (the one you usually persist)

For most apps the only persistence decision is whether your author is durable. `createAuthorIdentity({ persistAs: 'me' })` (5.2) handles it – no adapter, no manual dump/load.

10.2 The PersistenceAdapter (peer state)

To persist *peer* state – the node identity envelope, the synaptome, the active subscription list – pass a `PersistenceAdapter` as the constructor's `persist` option. The kernel ships two, behind sub-path imports so neither pulls `indexedDB` into Node nor `node:fs` into the browser bundle:

```
import { IndexedDBPersistence } from '@axona/protocol/persistence/indexeddb.js';
// import { FilePersistence } from '@axona/protocol/persistence/file.js'; // Node

const peer = new AxonaPeer({
  domain, node, nodeIdentity, transport,
  persist: new IndexedDBPersistence({ dbName: 'my-app' }),
});
await peer.start();
```

On `start()` the peer loads the node identity (if you did not pass one), the synaptome seed, and the subscription list (exposed as `peer.pendingSubscriptions` for you to re-register handlers – functions do not serialize). Writes are debounced; `leave()` force-flushes.

10.3 The node identity envelope (rarely)

If you want a stable Node ID across restarts (rare – see 4.2), round-trip the node identity through `dumpIdentity` / `loadIdentity`:

```
import { dumpIdentity, loadIdentity } from '@axona/protocol';

localStorage.setItem('node', JSON.stringify(await dumpIdentity(nodeIdentity)));
const restored = await loadIdentity(JSON.parse(localStorage.getItem('node')));
```

`loadIdentity` verifies the stored Node ID is internally consistent and that the private key actually corresponds to the public key, throwing on corruption.

10.4 Manual snapshots

```
const state    = await peer.snapshot();   // { identity?, subscriptions, synaptome, ... }
const restored = await AxonaPeer.fromSnapshot(state, { engine, node, transport });
```

`fromSnapshot` is a static factory; the restored peer is constructed and pre-loaded but not connected – call `peer.join()` to bring the transport up.

---

11. The bridge (Docker production)

The bridge does three things:

1. **Signaling for WebRTC** – peers exchange SDP offers/answers/ICE through it on a per-pair basis.
2. **Peer-list distribution** – a connecting peer gets the current peer list so it can initiate WebRTC offers.
3. **Universal-hub peer** – the bridge runs its own peer, meshed to every connected browser, acting as a routing hop for peers that cannot reach each other directly.

11.1 Using the public bridges

Point at the network for the line you're building on:

```
wss://testnet.axona.net        # the 4.x line (this guide) - kernel 4.11.2
wss://bridge.axona.net         # production east - still the 3.x line
wss://bridge-west.axona.net    # production west - still the 3.x line
```

The 4.x line runs on **testnet**; the production federated pair is still on 3.x, and the wire major partitions them (a 4.x peer and a 3.x bridge reject each other at the handshake). Open a WebSocket (the `webTransport` factory does the handshake for you) and you are on the network. A bridge advertises itself in the public bridge directory so clients can discover and fail over between bridges.

11.2 Running your own bridge (Docker stack)

Production runs a **three-container Docker stack** – this is the current deployment, not the old `npm start` + systemd path:

- `bridge` – the Node bridge (internal only; not published directly).
- `caddy` – a reverse proxy with **automatic HTTPS** (Let's Encrypt) that terminates TLS and proxies `wss://$DOMAIN` to the bridge.
- `coturn` – a TURN relay on host networking (it needs the full UDP relay port range and the real client source IPs, which Docker's bridge network would hide). Always deploy TURN with a bridge – a STUN-only bridge cannot relay for peers behind symmetric NAT.

```
git clone https://github.com/axona-net/axona-bridge
cd axona-bridge
cp .env.docker.example .env
$EDITOR .env            # set DOMAIN, PUBLIC_IP, TURN_AUTH_SECRET (and BRIDGE_PUBLIC_URL)
docker compose up -d --build
curl https://$DOMAIN/healthz
```

Prerequisites: a Linux Docker host with ports 80/443 free and DNS pointing at it. The full recipe (including the systemd/`install.sh` alternative, which is the simpler path if you need `turns://` TLS on the TURN port) is in `axona-bridge/deploy/README.md`.

11.3 Bridge environment (the ones that matter)

Var	Purpose
`DOMAIN`	The bridge's public hostname (Caddy provisions a cert for it).
`PUBLIC_IP`	The host's public IPv4, for coturn's `--external-ip`.
`TURN_AUTH_SECRET`	Shared static-auth secret; the bridge mints time-bound TURN creds with it and coturn validates against it. Must match.
`BRIDGE_PUBLIC_URL`	`wss://<this-bridge>` so it advertises itself in the bridge directory.

Var	Purpose
`BRIDGE_DIRECTORY`	`off` to opt a private/testnet bridge out of the public directory.
`BRIDGE_UPSTREAMS`	Comma-separated bridges to federate into (defaults to the public east/west pair).
`BRIDGE_LAT` / `BRIDGE_LNG` / `BRIDGE_REGION_LABEL`	The bridge's own region.
`PORT` / `LOG_LEVEL`	Internal port (8080) and log level.

A persistent `bridge-data` volume holds the bridge's identity keypair, so its Node ID survives restarts.

11.4 Federation

When `BRIDGE_DIRECTORY` is on, a bridge also bootstraps *into* the live mesh as a node (an outbound uplink to a known bridge) so its directory entry is visible network-wide. `bridge.axona.net` and `bridge-west.axona.net` are federated this way. `/healthz` exposes `uplink.{ upstream, connected }` and `directory.{ enabled, url }` so you can confirm what is live.

11.5 Surface the kernel version

Every Axona deployment should expose `KERNEL_VERSION` at a runtime-inspectable surface so it is obvious which kernel is live: the bridge at `/healthz`, an app in its version row, a Node peer in a startup log. Import the constant rather than hard-coding a string.

---

12. Worked example: a regional chat app

Let's build **AxonaTalk**: users join named rooms in a shared region, see history, and can DM each other. This follows the real pattern in `apps/axona-minimal/`.

12.1 The plan

- **Rooms** are **open topics** pinned to one region, so every participant derives the same Topic ID regardless of where they sit: `{ region: 'useast', name: room }`.
- **Each user** has a **durable author** (`persistAs`) so their authorship is stable across reloads, and an **ephemeral node identity** rooted at their real location.
- The protocol never reveals where a sender is. If we want to *show* a sender's region, the app voluntarily includes its own Node ID in the payload – an app choice, not a protocol disclosure.
- **DMs** ride `peer.send`, not pub/sub.

12.2 Message shape

```
// A chat message (the `message` field of the envelope):
{ text: 'hi', node: '<my Node ID hex>' }   // `node` is voluntary region disclosure

// A DM (sent via peer.send):
{ kind: 'axonatalk:dm', from: '<my Author ID>', text: '...', ts: 1779... }
```

12.3 Connect

```
import {
  AxonaPeer, AxonaDomain, NeuronNode,
  createNodeIdentity, createAuthorIdentity,
  deriveTopicId, regionName, regionCenter, KERNEL_VERSION,
} from '@axona/protocol';
```

```
import { webTransport } from '@axona/protocol/transport/web/index.js';

const BRIDGE       = 'wss://testnet.axona.net';  // the 4.x line runs on testnet (prod is 3.x)
const TOPIC_REGION = 'useast';                   // rooms are pinned here for everyone

let peer, nodeIdentity, author;

// Real geolocation -> the user's actual S2 cell; denied -> the room region.
function whereAmI() {
  const fallback = regionCenter(TOPIC_REGION);
  if (!navigator.geolocation) return Promise.resolve(fallback);
  return new Promise((resolve) => {
    navigator.geolocation.getCurrentPosition(
      (p)  => resolve({ lat: p.coords.latitude, lng: p.coords.longitude }),
      ()   => resolve(fallback),
      { timeout: 8000, maximumAge: 600000 });
  });
}

async function connect() {
  const here = await whereAmI();

  // CONNECTION identity -- ephemeral, rooted at the user's real location.
  nodeIdentity = await createNodeIdentity({ lat: here.lat, lng: here.lng });

  // AUTHORSHIP identity -- durable, location-free, names the signer on every post.
  author = await createAuthorIdentity({ persistAs: 'axonatalk:author' });

  const transport = webTransport({ bridgeUrl: BRIDGE, identity: nodeIdentity });
  const node = new NeuronNode({ id: BigInt('0x' + nodeIdentity.id), lat: here.lat, lng: here.lng });
  node.transport = transport;
  peer = new AxonaPeer({ domain: new AxonaDomain({ k: 20 }), node, nodeIdentity, transport });

  await transport.start(nodeIdentity.id);
  await peer.start();

  // Wait for the mesh to form before publishing (see pitfall 13.3).
  const until = Date.now() + 30_000;
  while (Date.now() < until && (node.synaptome?.size ?? 0) < 3) {
    await new Promise((r) => setTimeout(r, 600));
  }
  console.log(`AxonaTalk on kernel ${KERNEL_VERSION}; you are ${idLabel(nodeIdentity.id)}`);
  return { peer, author, nodeIdentity };
}

// "region:idprefix" read off a 264-bit Node ID we VOLUNTARILY shared in the
// payload -- top byte = the sender's S2 region. The protocol itself never
// carries this. Anonymous / absent -> 'anon'.
const idLabel = (nodeId) =>
  (typeof nodeId === 'string' && nodeId.length >= 10)
    ? `${regionName(parseInt(nodeId.slice(0, 2), 16))}:${nodeId.slice(2, 10)}`
    : 'anon';
```

12.4 Join a room

```
let currentRoom = null, currentSub = null;
const seen = new Set();   // msgIds already shown (dedup our own echo)

async function joinRoom(room, onMessage) {
  if (room === currentRoom) return;
  if (currentSub) { try { await currentSub.stop(); } catch {} }
  currentRoom = room;

  currentSub = await peer.sub({ region: TOPIC_REGION, name: room }, (env) => {
    if (!env || env.deleted || seen.has(env.msgId)) return;
    seen.add(env.msgId);
    const m = env.message;
    onMessage({
      text: (m && typeof m === 'object') ? m.text : m,
      who:  idLabel((m && typeof m === 'object') ? m.node : null),
    });
  }, { since: 'all' });   // give new joiners the room history
}
```

12.5 Send a chat message

```
async function sendChat(room, text) {
  await joinRoom(room, renderIncoming);   // make sure we're subscribed
  // Every publish names its signer. We voluntarily include our Node ID so
  // subscribers can show our region; the signed envelope never carries it.
  const msgId = await peer.pub(
    { region: TOPIC_REGION, name: room },
    { text, node: nodeIdentity.id },
    { signWith: author });
  seen.add(msgId);                        // our own publish may not echo back
  renderIncoming({ text, who: idLabel(nodeIdentity.id), self: true });
  return msgId;
}
```

12.6 Show a room's Topic ID (shareable read handle)

```
async function roomLink(room) {
  // deriveTopicId is pure -- works before we even connect.
  const id = await deriveTopicId({ region: TOPIC_REGION, name: room });
  return `${regionName(0x89)} : ${id}`;   // share this so others can sub by ID
}
```

12.7 Direct messages

```
async function sendDM(recipientNodeIdHex, text) {
  return peer.send(recipientNodeIdHex, {
    kind: 'axonatalk:dm', from: author.authorId, text, ts: Date.now(),
  });
}

function registerDMHandler(onDM) {
  peer.onMessage(async (senderId, message) => {
    if (message?.kind === 'axonatalk:dm') {
```

```
      onDM({ fromNode: senderId, fromAuthor: message.from, text: message.text });
      return { ack: true, receivedAt: Date.now() };
    }
  });
}
```

12.8 What you did and did not have to build

You did **not** build: a server (the bridge is shared infra), replay/history (`since: 'all'`), a message bus (the synaptome routes), or cryptography (`signWith` signs, `verifyEnvelope` checks).

You **did** decide: what an author means in your social model, your trust policy (who you accept by Author ID), and the UI. None of that is the protocol's job.

---

13. Common pitfalls

13.1 Forgetting `signWith`

`peer.pub` has **no default signer.** Omitting `signWith` throws `PUBLISH_NO_PUBLISH_IDENTITY` – it does not silently publish anonymously. Pass `{ signWith: author }`, or `{ signWith: ANONYMOUS }` if you really mean unsigned. (This trips up everyone migrating from the v2 line, where the node key signed by default.)

13.2 Publishing with a Topic ID

`pub` and `kill` need the **descriptor** – a bare 66-hex Topic ID throws `PUBLISH_INVALID_TOPIC`. Only `sub`, `pull`, and `metrics` accept an ID. Share the ID for reading; share the descriptor (`{ region, name }` or the full owned descriptor) for writing (6.6).

13.3 Publishing before the mesh forms

`peer.start()` returns as soon as local state is set up; mesh handshakes complete asynchronously over the next few seconds. Publish too early and your K-closest set is just yourself + the bridge, so the axon set is incomplete. The 4.x **cold-publish burst** (§7.1) now re-sends a freshly-joined node's first publishes over the first second, so an early publish usually still lands — this is a soft edge, not the hard 0% it once was. Still, for predictable UX, wait for `node.synaptome.size` (or `peer.peers().length`) to reach a small threshold before letting the user publish – the worked example polls for 3.

13.4 Mismatched descriptors give no delivery, silently

Publisher and subscriber must resolve to the **same Topic ID.** If one names `region: 'useast'` and the other omits `region` (defaulting to a different node region), or one adds an `owner` the other does not, the IDs differ and nothing is delivered – with no warning. Centralize your descriptor construction in one helper and call it from both sides.

13.5 Owned-topic write policy

Publishing to `{ owner, name }` (write defaults to `owner`) with a `signWith` that is not the owner key throws `WRITE_POLICY_VIOLATION` (the peer fails fast; the storing node enforces the same at ingress). If you meant an inbox anyone can post to, set `write: 'open'` explicitly.

13.6 Replay needs the message to still be cached – and someone to host it

`since: 'all'` returns whatever is in the axons' caches *right now.* If a topic has no subscribers and no hosting node, its role gets swept after a short grace window and the cache vanishes – even before the hold time elapses. For durable history, run a hosting node: `peer.host()` on a long-lived peer (or use `axona-relay`), or mirror envelopes into your own database at the app layer.

13.7 Message size

The reliable-publish floor is **16 KiB** on the serialized envelope. `peer.pub` throws `PUBLISH_PAYLOAD_TOO_LARGE` above it rather than letting an unreceivable message vanish mid-mesh. Do not ship blobs over pub/sub even under the cap – every publish replicates to K axons and fans to all subscribers. Publish a small content reference and move the bytes out of band, or use `@axona/protocol/std/chunk`.

13.8 Node identity is ephemeral; persist the author, not the node

A fresh `createNodeIdentity` each run means a new Node ID each run – by design (4.2). If your app needs stable identity for "block this user" lists or persistent threads, that stability comes from the **author** key (`persistAs`), which is location-free and is the recognizable thing. Persist the node identity only if you specifically want a stable address (rare).

13.9 `verifyEnvelope` returns an object

Test `.ok`. `if (await verifyEnvelope(env))` is always truthy and would never reject a forgery (5.5).

13.10 Hard-reload after a deploy

Browser bundle cache is sticky. After deploying a new peer, hard-reload (Cmd+Shift+R / Ctrl+Shift+R). If tabs run different versions across a wire change, the mesh half-works and debugging is miserable. The bridge rejects peers below its minimum version with an upgrade close code, surfaced via `peer.onUpgradeRequired`.

---

14. Production checklist

Before launching a public Axona app:

- ☐ **Docker bridge stack** – `docker compose up -d --build` with `DOMAIN`, `PUBLIC_IP`, `TURN_AUTH_SECRET`, `BRIDGE_PUBLIC_URL` set. Caddy gives you automatic HTTPS; do not hand-roll TLS.
- ☐ **TURN alongside the bridge** – coturn with a shared `TURN_AUTH_SECRET`. Not optional: symmetric-NAT peers cannot connect without it.
- ☐ **Bridge directory / federation** – set `BRIDGE_PUBLIC_URL` so the bridge advertises itself; set `BRIDGE_DIRECTORY=off` for a private fleet. Verify `/healthz` shows the directory and uplink state you expect.
- ☐ **Stable bridge identity** – the persistent `bridge-data` volume keeps the bridge's Node ID across restarts.
- ☐ **Author persistence** – `createAuthorIdentity({ persistAs })` so users keep a recognizable authorship across sessions.
- ☐ **Descriptor-based topic naming** – decide and document your topic shapes: which are open lobbies `{ region, name }`, which are owned feeds `{ region, owner, name }`, which are inboxes (`write: 'open'`). Build every descriptor through one shared helper so pub and sub never drift (13.4).
- ☐ **Region policy** – decide whether topics pin to one named region (for a shared keyspace) or default to the node region, and apply it consistently.
- ☐ **Signature / trust policy** – decide which Author IDs you trust and reject the rest in your handler (or accept everyone). Verify with `verifyEnvelope`.
- ☐ **Durable history** – if you need history beyond the cache window, run a hosting node (`peer.host()` / `axona-relay`) or journal envelopes to your own store (13.6).
- ☐ **Surface `KERNEL_VERSION`** – in the app version row and at the bridge `/healthz`, imported from the kernel (11.5).
- ☐ **Health monitoring** – scrape `https://<bridge>/healthz` and/or hook `peer.health().meshDegraded` into your dashboards.

---

15. Cheat sheet

All imports are `from '@axona/protocol'` unless a sub-path is shown.

15.1 Identity

```
createNodeIdentity({ lat, lng, extractable? })   // -> connection identity (Node ID)
createAuthorIdentity({ persistAs?, store?, extractable? })  // -> author identity (Author ID)
ANONYMOUS                                        // sentinel for an unsigned publish

dumpIdentity(nodeIdentity)  / loadIdentity(env)    // node-identity persistence envelope
```

15.2 Peer

```
new AxonaPeer({ domain, node, nodeIdentity, transport, persist? })   // NO author key here
await peer.start();  await peer.stop();
await peer.join(sponsorHex?);  await peer.leave({ drain?, notify?, timeoutMs? });
peer.getNodeId();   // 66-hex Node ID
```

15.3 Topics

```
deriveTopicId({ region?, owner?, name, write? })   // -> 66-hex read handle
// descriptor: region = name | '0x89' | 137 | omit(node region); owner = Author ID;
//             write = 'open' | 'owner' (defaults: owner present -> 'owner', else 'open')
```

15.4 Pub/Sub

```
peer.pub(descriptor, message, { signWith })        // -> msgId   (signWith REQUIRED)
peer.sub(descriptor | topicId, handler, { since? }) // -> Subscription; handler gets envelope or { msgI
peer.unsub(descriptor)                              // -> { ok, removed }
peer.kill(descriptor, msgId, { signWith })          // author-only retract (sole retraction primitive;
peer.pull(msgId | null, { topic, timeoutMs? })      // topic = descriptor | id; null -> latest
peer.metrics(descriptor | topicId, { timeoutMs? })  // one-shot read of the published snapshot (open +
metricTopic(dataTopicId)                            // -> metric topic descriptor; sub() it for live +
peer.sub(metricTopic(await deriveTopicId(desc)), cb, { since:'all' })   // scalable metrics
peer.host(descriptor?)  / peer.unhost(descriptor?)  // host()/unhost() = keyspace
peer.rootedTopics()                                 // infra: topics I root + local snapshots
await subscription.stop();
// since: omit (live) | 'all' | 'latest' | <ms epoch>
```

15.5 Direct messaging

```
peer.send(nodeIdHex, message)    // -> remote handler's return value
peer.notify(nodeIdHex, message)  // fire-and-forget
peer.onMessage((senderId, message) => reply)   // single handler
```

15.6 Introspection

```
peer.peers();                    // -> Node ID[] in the synaptome
peer.onPeerJoin(h);  peer.onPeerLeave(h);   // -> unsubscribe()
peer.lookup(targetBigInt);       // -> { found, hops, time, path }
peer.health();                   // -> diagnostic snapshot (meshDegraded, transport, axonRoles, ...)
peer.onLog(level, h);  peer.onError(h);  peer.onUpgradeRequired(h);
```

15.7 Envelope + region helpers

```
verifyEnvelope(envelope)         // -> { ok, reason?, signed }  (test .ok, not truthiness)
regionName(code);  regionCode(name);  resolveRegion(token);  regionCenter(nameOrCode);
```

15.8 Persistence (sub-path imports)

```
import { IndexedDBPersistence } from '@axona/protocol/persistence/indexeddb.js';
import { FilePersistence }      from '@axona/protocol/persistence/file.js';
```

15.9 Version constants

```
WIRE_VERSION         // '4.0'
KERNEL_VERSION       // '4.11.2'
```

15.10 Error codes worth catching

Errors subclass `AxonaError` with a stable `.code` – switch on `.code`, not `.message`:

Code	When
`PUBLISH_NO_PUBLISH_IDENTITY`	`pub` called without `signWith` (13.1)
`PUBLISH_INVALID_TOPIC`	a bare Topic ID passed where a descriptor is required (13.2)
`WRITE_POLICY_VIOLATION`	non-owner publishing to an owned topic (13.5)
`PUBLISH_PAYLOAD_TOO_LARGE`	enveloped message over the 16 KiB floor (13.7)
`PUBLISH_INVALID_MESSAGE`	message not JSON-serializable
`TOPIC_REGION_REQUIRED`	region omitted and no node region available
`UPGRADE_REQUIRED`	peer below the bridge's minimum wire version (13.10)

Where to go next

- **Quick Start** – a five-minute roundtrip for someone you are onboarding.
- **API Reference** – the exact signature of every public symbol.
- **Services Guide** – the programs you *run* rather than write: the signaling bridge, the relay + its front-ends (console, CLI, MCP server, desktop), directory/federation, and the PoW collector.
- **Identity & Authorship Model** – the design rationale behind the three-primitive model.
- **Read `apps/axona-minimal/`** in the kernel repo – a complete, runnable v3 app this guide's worked example is based on.
- **Read the security changelog** – the versioned record of what the protocol secures.

Found a bug or a gap in this guide? Open an issue at https://github.com/axona-net/axona-docs.